

TABLE OF CONTENT

I.INTRODUCTION.....	2
II.C PROGRAM STRUCTURE AND EXAMPLES.....	2
A. Basic Hello World Program.....	2
Figure 1: Basic hello world program.....	2
B. Program With User Input.....	3
Figure 2: Program with user input.....	3
C. Program With Formatted Output.....	4
Figure 3:Program with formatted output.....	4
III. GCC COMPILATION PROCESS.....	4
A. Installing GCC In Linux.....	4
B. Four Stages Of Compilation.....	5
C. Compilation Stages.....	5
Figure 5: preprocessing stage.....	6
Figure 6: compilation stage.....	7
Figure 8: Linking stage.....	8
Figure 9: program with return value.....	8
Figure 10: checking program exit status.....	9
V. KNOWLEDGE CHECK QUESTION.....	9
VI. LAB TASK COMPLETED.....	11
Figure 11: compiled program and verified exit status.....	11
Figure 12: modified program and return 1.....	12
Figure 13: program that accept input and display.....	12
Figure 14: compiled and run program returning value 50.....	16
VIII. CONCLUSION.....	17

Lab 2 : C Programming Basics

Lab Report and Knowledge Check

I.INTRODUCTION

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure compilation stages, and basic Linux commands for software development.

II.C PROGRAM STRUCTURE AND EXAMPLES

A. *Basic Hello World Program*

The following demonstrates a basic C program structure with preprocessor directives, main function , and return statement:

A screenshot of a terminal window with a dark purple background. The text is displayed in a monospaced font with syntax highlighting: '#include' is blue, '<stdio.h>' is orange, 'int' is green, 'main()' is green, '{' is green, 'printf' is green, '"Hello, World!\n"' is orange, ';' is green, 'return' is green, '0;' is green, and '}' is green. A white cursor is visible on the line following the closing brace.

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Figure 1: Basic hello world program

B. Program With User Input

This example demonstrates the use of printf and scanf for input/output operations:

```
#include <stdio.h>

int main() {
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 2: Program with user input

C. Program With Formatted Output

Demonstrating multiple format specifiers and variables:

```
GNU nano 8.7.1                                     hello.c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m):");
    scanf("%f", &height);

    printf("\n--- User Information ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 3: Program with formatted output

III. GCC COMPILATION PROCESS

A. Installing GCC In Linux

The following commands are used to install and verify GCC:

```
sunil@Ubuntu:~/lab2$
sunil@Ubuntu:~/lab2$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

sunil@Ubuntu:~/lab2$
```

Figure 4: GCC version check

B. Four Stages Of Compilation

The C Compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

Table 1
The Four Stages of Compilation

Stage	Command	Output	Purpose
1. Preprocessing	<code>gcc -E hello.c -o hello.i</code>	<code>.i</code> file	Expand headers and macros
2. Compiling	<code>gcc -S hello.i -o hello.s</code>	<code>.s</code> file	Generate assembly code
3. Assembling	<code>gcc -C hello.s -o hello.o</code>	<code>.o</code> file	Create object file
4. Linking	<code>gcc -E hello.c -o hello</code>	executable	Final executable

C. Compilation Stages

Preprocessing

Commands to expand all headers and macros and view the preprocessed output.

```
sunil@Ubuntu:~/lab2$ gcc -E hello.c -o hello.i
sunil@Ubuntu:~/lab2$
sunil@Ubuntu:~/lab2$ cat hello.i | less

[1]+  Stopped                  cat hello.i | less
sunil@Ubuntu:~/lab2$
```

```
sunil@Ubuntu: ~/lab2
~/lab2

# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
```

Figure 5: preprocessing stage

Compilation to Assembly

Command to generate assembly code and view assembly code.

```
sunil@Ubuntu:~/lab2$  
sunil@Ubuntu:~/lab2$ gcc -S hello.i -o hello.s  
sunil@Ubuntu:~/lab2$ cat hello.s  
.file "hello.c"  
.text  
.section .rodata  
.LC0:  
.string "Enter your name: "  
.LC1:  
.string "%s"  
.LC2:  
.string "Enter your age: "  
.LC3:  
.string "%d"  
.LC4:  
.string "Enter your height (m):"  
.LC5:  
.string "%f"  
.LC6:  
.string "\n--- User Information ---"  
.LC7:  
.string "Name: %s\n"  
.LC8:  
.string "Age: %d years\n"  
.LC9:  
.string "Height: %.2f m\n"  
.text  
.globl main  
.type main, @function  
main:  
.LFB0:  
.cfi_startproc  
endbr64  
pushq %rbp  
.cfi_def_cfa_offset 16  
.cfi_offset 6, -16  
movq %rsp, %rbp  
.cfi_def_cfa_register 6  
subq $80, %rsp  
movq %fs:40, %rax  
movq %rax, -8(%rbp)  
xorl %eax, %eax  
leaq .LC0(%rip), %rax  
movq %rax, %rdi  
movl $0, %eax  
call printf@PLT
```

Figure 6: compilation stage

Assembly to Object File

Command to create object file and check file type:

```
sunil@Ubuntu:~/lab2$  
sunil@Ubuntu:~/lab2$ gcc -c hello.s -o hello.o  
sunil@Ubuntu:~/lab2$ file hello.o  
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped  
sunil@Ubuntu:~/lab2$
```

Figure 7 : Assembly stage

Linking

Commands to create file executable and run the program.

```
sunil@Ubuntu:~/lab2$ gcc hello.o -o hello
sunil@Ubuntu:~/lab2$ ./hello
Enter your name: sunil
Enter your age: 23
Enter your height (m):170

--- User Information ---
Name: sunil
Age: 23 years
Height: 170.00 m
sunil@Ubuntu:~/lab2$
```

Figure 8: Linking stage

IV. RETURN VALUES AND EXIT STATUS

A. Understanding Return values

```
#include <stdio.h>

int main () {
    printf("Program executed successfully\n");
    return 0;
}
```

Figure 9: program with return value

B. Checking Exit Status

```
sunil@Ubuntu:~/lab2$ gcc return.c -o return
sunil@Ubuntu:~/lab2$ ./return
Program executed successfully
sunil@Ubuntu:~/lab2$

sunil@Ubuntu:~/lab2$ nano return.c
sunil@Ubuntu:~/lab2$ echo $?
0
sunil@Ubuntu:~/lab2$ nano return.c
sunil@Ubuntu:~/lab2$ gcc return.c -o return
sunil@Ubuntu:~/lab2$ ./return
Program executed successfully
sunil@Ubuntu:~/lab2$ echo $?
1
sunil@Ubuntu:~/lab2$
```

Figure 10: checking program exit status

V. KNOWLEDGE CHECK QUESTION

A. Question 1

Q: What is the purpose of the #include directive in C programming? Provide an example of its usage.

=>> It is used to add header files in the c program in the preprocessing section.

For example:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

B. Question 2

Q. Explain the four stages of compilation. What file extension is produced at each stage?

=>> The four stages of compilation are :

- Preprocessing : It processes directives such as #include. Its file extension is .i.
- Compiling : It converts preprocessed C code into assembly code. Its file extension is .s
- Assembling: It converts assembly code to machine code. Its file extension is .o
- Linking: It combines object files and libraries to create the executable.

C. Question 3

Q: What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

=>> A return value of 0 from main() indicates that the program completed successfully without errors. A non-zero return value generally indicates that the program encountered an error or terminated unsuccessfully.

D. Question 4

Q: Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the user

=>>

```
#include <stdio.h>

int main() {
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

E. Question 5

Q: What GCC command would you use to:

1) Only preprocess a file without compiling?

=>> gcc -E program.c -o program.i

2) Generate assembly code but not assemble it?

=>> gcc -S program.c -o program.s

3) Create an object file without linking

=>> gcc -c program.c -o program.o

F. Question 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

=>> C is called a compiled language because the C source code is converted into machine code by a compiler such as GCC before the program is executed.

VI. LAB TASK COMPLETED

1) Task 1: Compiled hello.c with return 0 and verified exit status using echo \$?

```
#include <stdio.h>

int main () {
    printf("Program executed successfully\n");
    return 0;
}
```

```
sunil@Ubuntu:~/lab2$ gcc return.c -o return
sunil@Ubuntu:~/lab2$ ./return
Program executed successfully
sunil@Ubuntu:~/lab2$
```

```
sunil@Ubuntu:~/lab2$ echo $?
0
```

Figure 11: compiled program and verified exit status

2) Task 2: Modified program to return 1 and observed different exit status

```
sunil@Ubuntu:~/lab2$ nano return.c
sunil@Ubuntu:~/lab2$ echo $?
0
sunil@Ubuntu:~/lab2$ nano return.c
sunil@Ubuntu:~/lab2$ gcc return.c -o return
sunil@Ubuntu:~/lab2$ ./return
Program executed successfully
sunil@Ubuntu:~/lab2$ echo $?
1
sunil@Ubuntu:~/lab2$
```

Figure 12: modified program and return 1

3) Task 3: Created program accepting user input and displaying it

```
#include<stdio.h>

int main () {
    int number;
    printf("Enter a number:");
    scanf("you entered: %d \n", &number);

    return 0;
}
```

```
sunil@Ubuntu:~/lab2$ gcc task3.c -o task3
sunil@Ubuntu:~/lab2$ ./task3
Enter a number:45
sunil@Ubuntu:~/lab2$
```

Figure 13: program that accept input and display

4) Task 4: Executed all four compilation stages separately and documented output files

Preprocessing

Commands to expand all headers and macros and view the preprocessed output.

```
sunil@Ubuntu:~/lab2$ gcc -E hello.c -o hello.i
sunil@Ubuntu:~/lab2$
sunil@Ubuntu:~/lab2$ cat hello.i | less

[1]+  Stopped                  cat hello.i | less
sunil@Ubuntu:~/lab2$
```



```
# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
```

Compilation to Assembly

Command to generate assembly code and view assembly code.

```
sunil@Ubuntu:~/lab2$  
sunil@Ubuntu:~/lab2$ gcc -S hello.i -o hello.s  
sunil@Ubuntu:~/lab2$ cat hello.s  
        .file "hello.c"  
        .text  
        .section      .rodata  
.LC0:  
        .string "Enter your name: "  
.LC1:  
        .string "%s"  
.LC2:  
        .string "Enter your age: "  
.LC3:  
        .string "%d"  
.LC4:  
        .string "Enter your height (m):"  
.LC5:  
        .string "%f"  
.LC6:  
        .string "\n--- User Information ---"  
.LC7:  
        .string "Name: %s\n"  
.LC8:  
        .string "Age: %d years\n"  
.LC9:  
        .string "Height: %.2f m\n"  
        .text  
        .globl main  
        .type main, @function  
main:  
.LFB0:  
        .cfi_startproc  
        endbr64  
        pushq %rbp  
        .cfi_def_cfa_offset 16  
        .cfi_offset 6, -16  
        movq %rsp, %rbp  
        .cfi_def_cfa_register 6  
        subq $80, %rsp  
        movq %fs:40, %rax  
        movq %rax, -8(%rbp)  
        xorl %eax, %eax  
        leaq .LC0(%rip), %rax  
        movq %rax, %rdi  
        movl $0, %eax  
        call printf@PLT
```

Assembly to Object File

Command to create object file and check file type:

```
sunil@Ubuntu:~/lab2$  
sunil@Ubuntu:~/lab2$ gcc -c hello.s -o hello.o  
sunil@Ubuntu:~/lab2$ file hello.o  
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped  
sunil@Ubuntu:~/lab2$
```

Linking

Commands to create file executable and run the program.

```
sunil@Ubuntu:~/lab2$ gcc hello.o -o hello
sunil@Ubuntu:~/lab2$ ./hello
Enter your name: sunil
Enter your age: 23
Enter your height (m):170

--- User Information ---
Name: sunil
Age: 23 years
Height: 170.00 m
sunil@Ubuntu:~/lab2$
```

5) Task 5: Compiled and ran program returning integer value 50

```
GNU nano 8.7.1
#include<stdio.h>

int main () {
    printf("program executed\n");
    return 50;
}

sunil@Ubuntu:~/lab2$ gcc task5.c -o task5
sunil@Ubuntu:~/lab2$ ./task5
program executed
sunil@Ubuntu:~/lab2$ echo $?
50
sunil@Ubuntu:~/lab2$
```

Figure 14: compiled and run program returning value 50

VIII. CONCLUSION

This laboratory provided practical experience with C programming fundamentals and the GCC compiler in a Linux environment. The exercises covered basic C program structure, user input and output, formatted output, compilation stages, and program exit status.

The practical work demonstrated the four stages of compilation: preprocessing, compilation, assembly, and linking. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions .

Through these activities, a better understanding of how C programs are written, compiled, executed, and communicated with the operating system was developed. These concepts provide an important foundation for further study of operating systems, system programming, and computer networks.